



Numerical Optimization

Winter semester 2025/26

Exercise sheet 8 for Numerical Optimization

Manuel Caipo

Degree program: M.Sc. Computational Science Engineering

Enrollment number: 30783212

Instructors: Dr. Michael Lehn, Tobias Born

02. December 2025 - 89081 Ulm

Chapter 1

Solutions Exercise sheet 8

1.1 Exercise 1

1.1.1 Part a)

The Core Idea

The main goal is to find a step vector, denoted by p , that achieves two objectives simultaneously:

- It must decrease the value of the objective function f .
- It must not move too far from the current iterate x_k .

This is precisely the purpose of a *trust region*: it defines a neighborhood around the current point x_k where we trust our model of the function to be a good approximation. The trust region is a constraint that says:

“You are allowed to move, but only within a maximum distance of Δ_k .”

This leads to the formulation of the trust-region subproblem, which is to find the step p that minimizes a quadratic model $q_k(p)$ of the function f subject to the trust-region constraint:

$$\min_{p: \|p\| \leq \Delta_k} q_k(p) \quad (1.1)$$

Here, $q_k(p)$ is a **quadratic approximation** of the function f at the point x_k . This model can be thought of as our best guess of what the function looks like in the immediate vicinity of our current position. where $q_k(p)$ is the quadratic model of f around x_k , defined as

$$q_k(p) = f(x_k) + \nabla f(x_k)^T p + \frac{1}{2} p^T H(x_k) p. \quad (1.2)$$

The model $q_k(p)$ approximates f “near” the point x_k using a second-order Taylor expansion. Solving this problem exactly is expensive, so we consider a simplified approach: we move only along the direction of steepest descent.

- Restricting the Search Direction The direction of steepest descent of f at x_k is given by the negative gradient $-\nabla f(x_k)$. Thus, we restrict the step p to the form

$$p = -\tau \nabla f(x_k), \quad (1.3)$$

where $\tau \geq 0$ determines how far we move in that direction. Let us define

$$g := \nabla f(x_k), \quad (1.4)$$

for shorter notation. Then the step becomes $p = -\tau g$. The optimization problem reduces to finding the best scalar τ :

$$\min_{0 \leq \tau \leq \tau_{\max}} q_k(-\tau g), \quad (1.5)$$

where $\tau_{\max}$ is determined by the trust-region constraint.

- Trust-Region Constraint The trust region requires

$$\|p\| \leq \Delta_k. \quad (1.6)$$

Since $p = -\tau g$, we obtain

$$\|-\tau g\| = \tau \|g\| \leq \Delta_k, \quad (1.7)$$

which implies

$$0 \leq \tau \leq \frac{\Delta_k}{\|g\|}. \quad (1.8)$$

Thus τ must lie within a bounded interval.

- Substituting the Step into the Quadratic Model We now substitute $p = -\tau g$ into $q_k(p)$:

$$q_k(-\tau g) = f(x_k) + g^T(-\tau g) + \frac{1}{2}(-\tau g)^T H(-\tau g) \quad (1.9)$$

$$= f(x_k) - \tau g^T g + \frac{1}{2} \tau^2 g^T H g. \quad (1.10)$$

This is a simple quadratic function in the scalar variable τ .

- Minimizing the Quadratic in τ Ignoring the trust-region bound for a moment, we minimize $q_k(-\tau g)$ by setting the derivative with respect to τ to zero:

$$\frac{d}{d\tau} q_k(-\tau g) = -g^T g + \tau g^T H g. \quad (1.11)$$

Setting the derivative to zero gives

$$-g^T g + \tau g^T H g = 0. \quad (1.12)$$

Solving for τ yields

$$\tau^* = \frac{g^T g}{g^T H g}. \quad (1.13)$$

This is the unconstrained minimizer of the quadratic model in the direction of steepest descent. However, the trust-region bound may force a smaller τ .

- The Role of the Curvature There are two cases depending on the sign of $g^T H g$:

Case 1: $g^T H g \leq 0$ (**Non-positive curvature**).

If $g^T H g \leq 0$, then the quadratic function in τ opens downward or is flat. In this case, there is no finite minimizer inside the interval $\tau \geq 0$; the function decreases as τ increases. Therefore, the best choice is on the boundary of the trust region:

$$\tau = \frac{\Delta_k}{\|g\|}, \quad (1.14)$$

and the Cauchy step becomes

$$p = -\frac{\Delta_k}{\|g\|} g. \quad (1.15)$$

Case 2: $g^T H g > 0$ (**Positive curvature**).

If $g^T H g > 0$, then the quadratic opens upward and the minimizer exists:

$$\tau^* = \frac{g^T g}{g^T H g}. \quad (1.16)$$

We must check whether this τ^* satisfies the trust-region constraint. If

$$\tau^* \leq \frac{\Delta_k}{\|g\|}, \quad (1.17)$$

then $\tau = \tau^*$ is feasible and optimal. Otherwise, τ must be truncated to the boundary:

$$\tau = \frac{\Delta_k}{\|g\|}. \quad (1.18)$$

Therefore, in the positive curvature case we obtain

$$\tau = \min \left(\frac{g^T g}{g^T H g}, \frac{\Delta_k}{\|g\|} \right). \quad (1.19)$$

The corresponding step is simply

$$p = -\tau g. \quad (1.20)$$

This step is known as the *Cauchy point*.

Combining both cases, the Cauchy point for the trust-region subproblem is

$$p = -\tau \nabla f(x_k), \quad (1.21)$$

where

$$\tau = \min \left(\frac{\|\nabla f(x_k)\|^2}{\nabla f(x_k)^T H(x_k) \nabla f(x_k)}, \frac{\Delta_k}{\|\nabla f(x_k)\|} \right). \quad (1.22)$$

This provides a simple and efficient way to obtain a feasible step that decreases the quadratic model while respecting the trust-region radius.

1.1.2 Part b)

```

1 function [X, Delta_hist] = trust_region_cauchy( ...
2     f, grad_f, H_f, ...
3     x0, tol, max_it, Delta0, ...
4     eta1, eta2, nu1, nu2)
5     xk = x0(:);
6     n = length(xk);
7     X = zeros(n, max_it + 1);
8     Delta_hist = zeros(max_it + 1, 1);
9
10    X(:,1) = xk;
11    Delta_hist(1) = Delta0;
12
13    k = 0; % contador de iteraciones (corresponde al x_k)
14
15    while k < max_it
16        k = k + 1; % vamos a construir x_k a x_{k+1}
17        gk = grad_f(xk);
18        gnorm = norm(gk);
19        if gnorm < tol
20            break;
21        end
22        Hk = H_f(xk);
23        gHg = gk' * Hk * gk;
24        Delta_k = Delta_hist(k);
25        if gHg <= 0
26            pk = - (Delta_k / gnorm) * gk;
27        else
28            alpha1 = (gnorm^2) / gHg;
29            alpha2 = Delta_k / gnorm;
30            alpha = min(alpha1, alpha2);
31            pk = - alpha * gk;
32        end
33        fk = f(xk);
34        fk_new = f(xk + pk);
35        act_red = fk - fk_new;
36        pred_red = - gk' * pk - 0.5 * pk' * Hk * pk;
37        if pred_red <= 0
38            rho = 0;
39        else
40            rho = act_red / pred_red;
41        end
42        if rho > eta1
43            x_next = xk + pk; % aceptamos
44        else
45            x_next = xk; % rechazamos
46        end
47        Delta_next = Delta_k;
48        if rho <= eta1
49            Delta_next = nu1 * Delta_k;
50        elseif (rho >= eta2) && (abs(Delta_k - norm(pk)) < 1e-12)
51            Delta_next = nu2 * Delta_k;
52        end
53        X(:, k+1) = x_next;
54        Delta_hist(k+1) = Delta_next;
55        xk = x_next;
56    end
57    X = X(:, 1:(k+1));
58    Delta_hist = Delta_hist(1:(k+1));
59 end

```

Trust Region Cauchy

1.1.3 Part b) c)

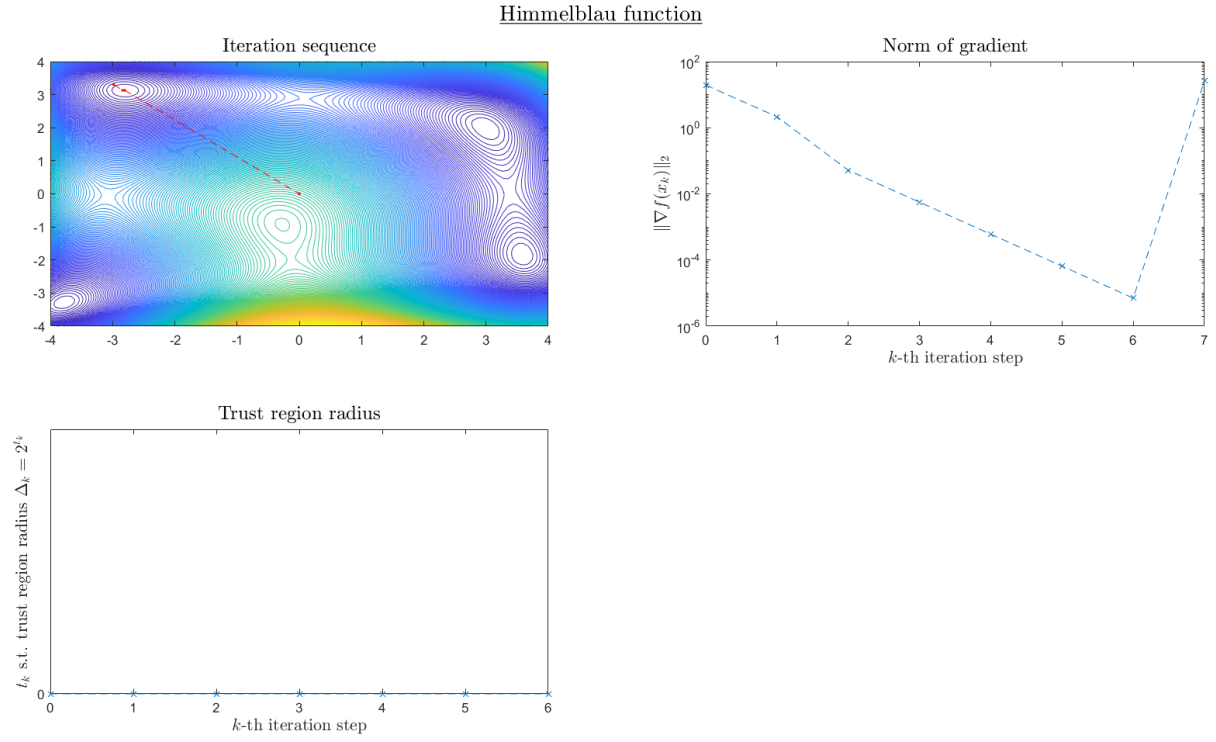


Figure 1.1: Himmelblau Comparison

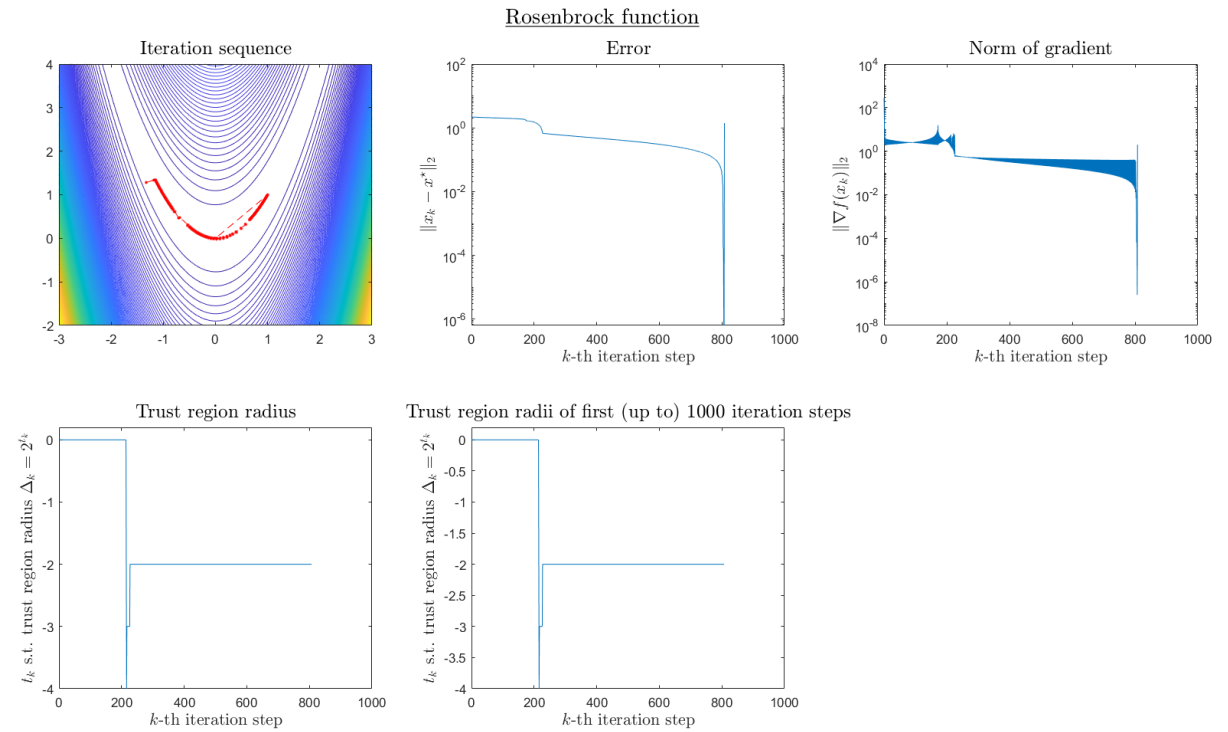


Figure 1.2: Rosenbrock Comparison

Role of the trust region radius At the k -th iteration the method solves the model minimization

$$\min_{\|p\| \leq \Delta_k} m_k(p) = f(x_k) + g_k^T p + \frac{1}{2} p^T H_k p,$$

where $\Delta_k > 0$ is a scalar that defines the size of the region in which the quadratic model $m_k(p)$ is assumed to be reliable. The trust-region radius does not multiply a direction, but acts as a constraint on the step length:

$$\|p_k\| \leq \Delta_k.$$

The update of Δ_k depends on the ratio

$$\rho_k = \frac{f(x_k) - f(x_k + p_k)}{-g_k^T p_k - \frac{1}{2} p_k^T H_k p_k},$$

which measures how well the local model predicts the actual decrease of f . If ρ_k is large, the quadratic model is accurate and Δ_k may be increased. If ρ_k is small, Δ_k is reduced.

Himmelblau function Starting from a random point, the norm of the gradient is initially large because Himmelblau has several well-separated local minima with steep slopes around them. As the iterates enter the basin of attraction of one minimum, $\|\nabla f(x_k)\|$ decreases rapidly and almost monotonically. In this case the quadratic model m_k is already reasonably accurate on the relevant region, so the ratio ρ_k remains acceptable and the trust-region radius stays nearly constant:

$$\Delta_{k+1} \approx \Delta_k.$$

The method reaches the minimum in a few steps and exhibits a simple trace on the contour plot.

Rosenbrock function The Rosenbrock function has a narrow, curved valley leading to its minimum. The gradient direction changes significantly along this valley, and the curvature in the directions of descent is highly anisotropic. As a consequence, the predicted decrease from $m_k(p)$ is often inaccurate when the iterate is not sufficiently close to the valley floor. For this reason the ratio ρ_k oscillates, and the method frequently adjusts the trust region radius:

$$\Delta_{k+1} \in \{\nu_1 \Delta_k, \Delta_k, \nu_2 \Delta_k\}.$$

These adjustments produce visible changes in $\|\nabla f(x_k)\|$ along the iterations. Far from the minimizer the iterations may zig-zag toward the valley, while close to the minimizer the method exhibits global convergence but only with linear rate due to the Cauchy step.

The trust-region Cauchy method reacts to each function according to the local geometry:

- On Himmelblau, the landscape is smooth near each minimum; the quadratic model is reliable, ρ_k is stable, and Δ_k does not change significantly.
- On Rosenbrock, the narrow and curved valley makes the quadratic model unreliable away from the minimum; the ratio ρ_k varies and the method adapts Δ_k frequently.

This behavior illustrates the central idea of trust-region methods: the step length is not predetermined, but is dynamically controlled based on the local accuracy of the model.

1.2 Exercise 2

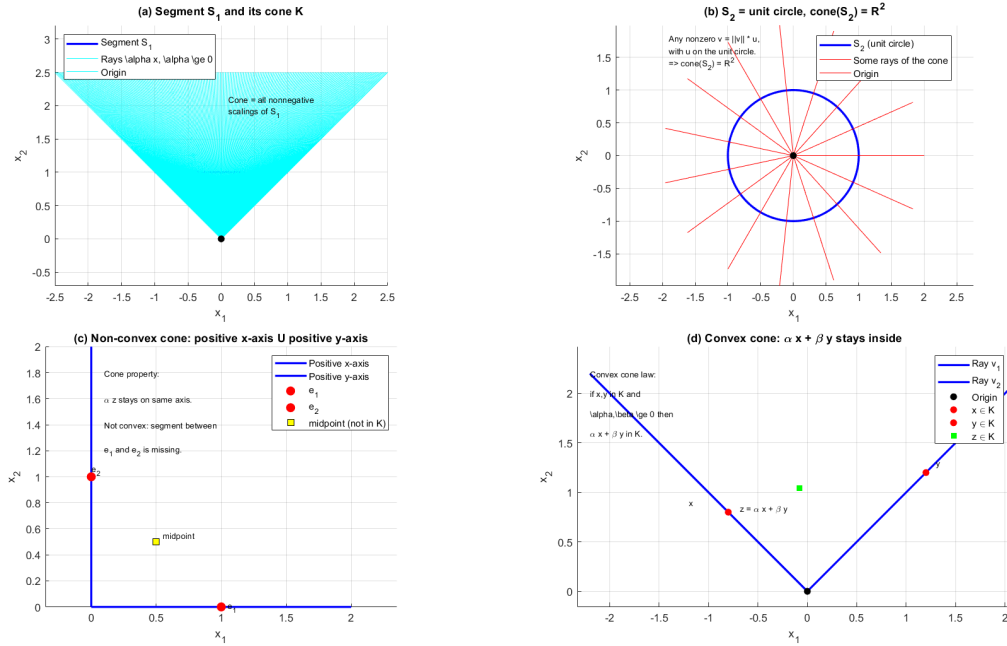


Figure 1.3: Cones evaluation

Cones and their role in optimization A set $K \subset \mathbb{R}^n$ is called a *cone* if it is closed under nonnegative scaling, that is

$$x \in K, \alpha \geq 0 \implies \alpha x \in K. \quad (1.23)$$

Geometrically, a cone consists of all rays starting at the origin that pass through a given generating set.

1.2.1 Part a)

Let $S_1 = \{\beta(-1, 1)^T + (1 - \beta)(1, 1)^T \mid \beta \in [0, 1]\}$ be a line segment. The cone generated by S_1 is

$$K = \{\alpha x \mid \alpha \geq 0, x \in S_1\}, \quad (1.24)$$

which corresponds to all nonnegative scalings of the segment.

1.2.2 Part b)

(Cone equal to $\mathbb{R}^2$.) Let $S_2 = \{x \in \mathbb{R}^2 \mid \|x\| = 1\}$ be the unit circle. Then every nonzero vector $v \in \mathbb{R}^2$ can be written as $v = \|v\| \cdot u$ with $u \in S_2$. Hence,

$$\text{cone}(S_2) = \mathbb{R}^2. \quad (1.25)$$

1.2.3 Part c)

(Non-convex cone.) The union of the positive coordinate axes,

$$K = \{(t, 0)^T \mid t \geq 0\} \cup \{(0, t)^T \mid t \geq 0\}, \quad (1.26)$$

is a cone since it is closed under nonnegative scaling. However, it is not convex, because for instance $(1, 0)^T$ and $(0, 1)^T$ belong to K , but their midpoint $(\frac{1}{2}, \frac{1}{2})^T$ does not.

1.2.4 Part d)

(d) Characterization of convex cones. A set K is a convex cone if and only if

$$\forall x, y \in K, \forall \alpha, \beta \geq 0 : \quad \alpha x + \beta y \in K. \quad (1.27)$$

Indeed, if K is a cone and convex, the property follows by combining closure under convex combinations and nonnegative scaling. Conversely, choosing $\beta = 0$ shows closure under scaling, and choosing $\alpha + \beta = 1$ shows convexity.

Importance in optimization. Cones arise naturally in constrained optimization to describe feasible directions at boundary points. The feasible direction cone characterizes all infinitesimal moves that remain inside the constraint set. In optimality conditions, the normal cone $N_C(x^*)$ appears in the Karush–Kuhn–Tucker conditions. Many modern algorithms, such as conic programming and trust-region methods with constraints, make essential use of conic geometry.

1.3 Exercise 3

1.3.1 Part a)

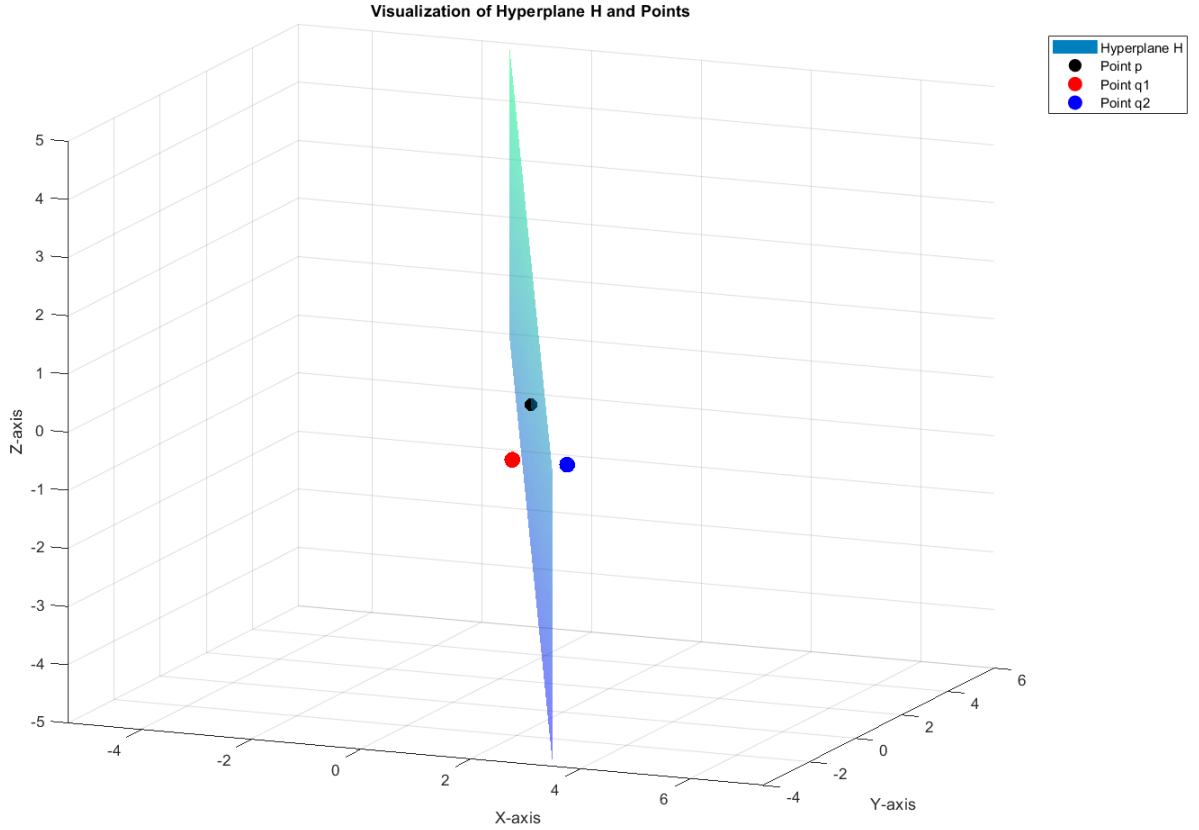


Figure 1.4: Hyperplane

Short Explanation of the Calculations What we did is a *membership test* for the hyperplane. The hyperplane is given in parametric form

$$H = \{ x \in \mathbb{R}^3 \mid x = p + s_1 u_1 + s_2 u_2, s_1, s_2 \in \mathbb{R} \}, \quad (1.28)$$

with a fixed starting point p and two direction vectors u_1, u_2 .

To check whether a point q lies on H , we try to solve

$$q = p + s_1 u_1 + s_2 u_2. \quad (1.29)$$

If such real numbers s_1, s_2 exist, then $q \in H$; if not, then $q \notin H$.

Writing this equation in coordinates gives a system of three scalar equations (one for each coordinate x, y, z):

$$\begin{aligned} q_x &= p_x + s_1(u_1)_x + s_2(u_2)_x, \\ q_y &= p_y + s_1(u_1)_y + s_2(u_2)_y, \\ q_z &= p_z + s_1(u_1)_z + s_2(u_2)_z. \end{aligned} \quad (1.30)$$

Point q_1 . For the point q_1 we first use the equations for the y and z coordinates and obtain that we must have

$$s_2 = 2. \quad (1.31)$$

However, if we insert $s_2 = 2$ into the equation for the x coordinate, we obtain

$$\frac{3}{2} = 2, \quad (1.32)$$

which is impossible. Therefore the system has no solution (s_1, s_2) and hence $q_1 \notin H$.

Point q_2 . For the point q_2 the same phenomenon occurs. From the y and z equations we again get the condition

$$s_2 = 2. \quad (1.33)$$

Plugging this into the x equation leads to

$$\frac{5}{2} = 2, \quad (1.34)$$

which is again a contradiction. Thus, the system has no solution and $q_2 \notin H$.

In summary, there is no combination of “steps” (s_1, s_2) that moves us from the starting point p to q_1 or q_2 using only the allowed directions u_1 and u_2 . Geometrically, both points lie outside of the hyperplane, which is why they appear to be “floating” off the plane in the plot.

The Importance of Hyperplanes in Optimization Hyperplanes are fundamental in optimization because they describe *boundaries* and *constraints*.

An optimization problem can be seen as the task of finding the “best” point (for example, minimal cost or maximal profit) in a multidimensional space. The rules of the problem (budgets, resource limits, physical laws, etc.) are often written as linear equations or inequalities.

A linear equation of the form

$$v^T x = c \quad (1.35)$$

defines a hyperplane. It models a strict constraint such as “you must spend exactly your entire budget”. All feasible solutions satisfying this rule lie on this hyperplane.

A linear inequality of the form

$$v^T x \leq c \quad (1.36)$$

defines a halfspace: all points on one side of the hyperplane, including the boundary $v^T x = c$. This is typical for constraints like “your spending must not exceed your budget”.

Hyperplanes are important for several reasons:

- **Defining the feasible region.** Hyperplanes and halfspaces carve out the set of all points that satisfy the constraints. Without them we would search for an optimum in an unstructured, infinite space.
- **Locating optimal points.** In linear programming, it is known that, if an optimal solution exists, it is attained at a *vertex* (corner) of the feasible region. These vertices are intersections of several constraint hyperplanes.
- **Separability and Farkas’ Lemma.** Farkas’ Lemma uses hyperplanes as *separating* objects. Roughly speaking, for certain systems, either there exists a solution, or there exists a hyperplane that separates the constraints from the desired point, proving that no solution is possible. Thus, a hyperplane can serve as a certificate that a problem is infeasible.

In short, hyperplanes are the geometric language of linear constraints. They allow us to visualize, analyze, and solve optimization problems by turning abstract rules into precise geometric boundaries.

1.3.2 Part b)

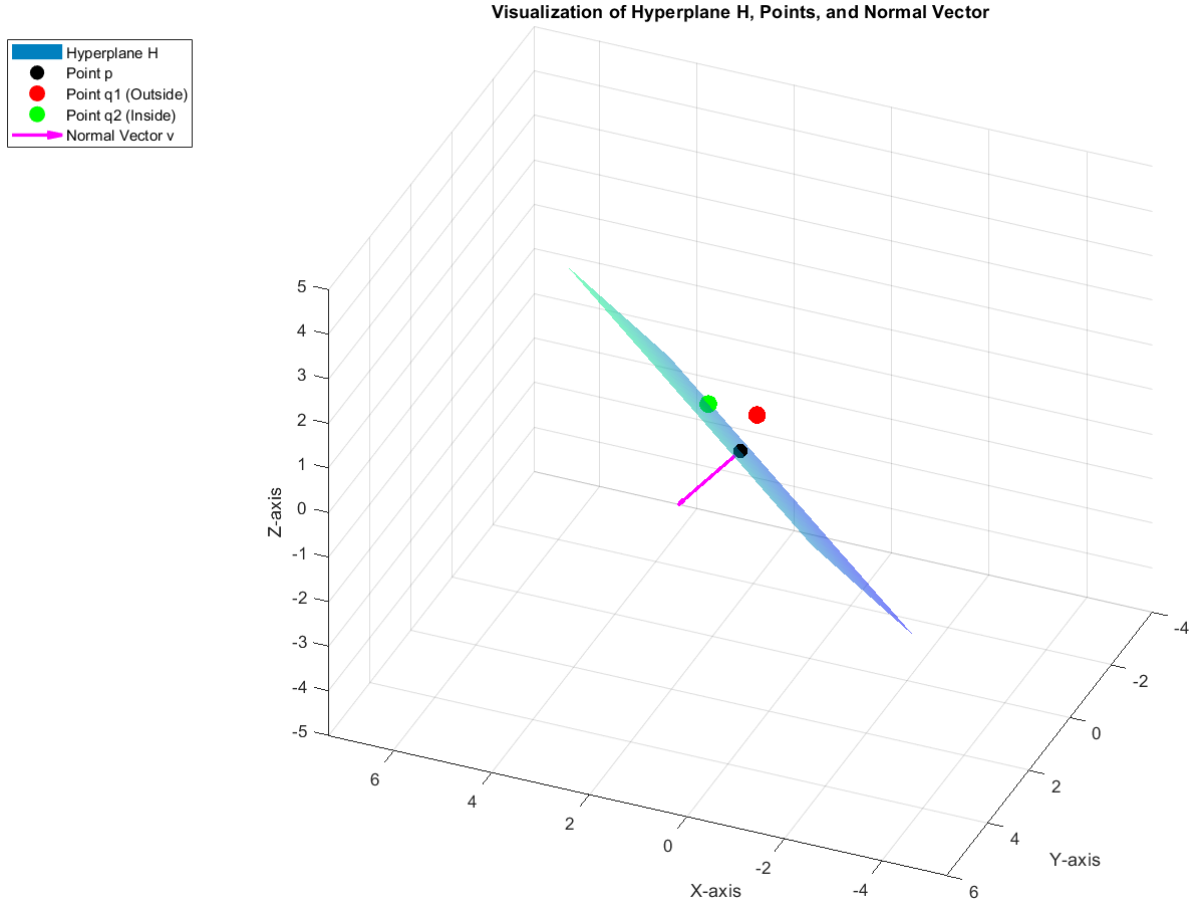


Figure 1.5: Hyperplane

The exercise asks us to convert the hyperplane H from its parametric form to its normal form, and then use the normal form to verify if points q_1 and q_2 lie on H .

The parametric form of the hyperplane is given by:

$$H = \{x \in \mathbb{R}^3 \mid x = p + s_1 d_1 + s_2 d_2\} \quad (1.37)$$

where $p = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$, $d_1 = \begin{pmatrix} 1/2 \\ 0 \\ 1/2 \end{pmatrix}$, and $d_2 = \begin{pmatrix} 0 \\ -1/2 \\ -1/2 \end{pmatrix}$.

The normal form of a hyperplane is given by:

$$H = \{x \in \mathbb{R}^n \mid v^T x = c\} \quad (1.38)$$

where v is a normal vector to the hyperplane and c is a constant.

- Finding the Normal Vector v The normal vector v is orthogonal to the direction vectors d_1 and d_2 . In $\mathbb{R}^3$, we can find such a vector by computing the cross product of d_1 and d_2 .

$$v = d_1 \times d_2 = \begin{pmatrix} 1/2 \\ 0 \\ 1/2 \end{pmatrix} \times \begin{pmatrix} 0 \\ -1/2 \\ -1/2 \end{pmatrix} \quad (1.39)$$

Calculating the components:

$$v = \begin{pmatrix} (0)(-1/2) - (1/2)(-1/2) \\ (1/2)(0) - (1/2)(-1/2) \\ (1/2)(-1/2) - (0)(0) \end{pmatrix} = \begin{pmatrix} 1/4 \\ 1/4 \\ -1/4 \end{pmatrix} \quad (1.40)$$

The normal form is unique up to the length of the normal vector. To simplify our calculations, we can use any non-zero scalar multiple of v . Let's multiply by 4 to get a simpler, integer-component vector.

$$v = 4 \begin{pmatrix} 1/4 \\ 1/4 \\ -1/4 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix} \quad (1.41)$$

- Finding the Constant c The constant c is found by computing $v^T p$, where p is any known point on the hyperplane. We use the point $p = (1, 1, 0)$ from the parametric definition.

$$c = v^T p = \begin{pmatrix} 1 & 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = (1)(1) + (1)(1) + (-1)(0) = 2 \quad (1.42)$$

Thus, the normal form of the hyperplane H is $v^T x = 2$, which can be written as the equation:

$$x_1 + x_2 - x_3 = 2 \quad (1.43)$$

- Checking if q_1 and q_2 lie on H We now check if the points q_1 and q_2 satisfy the normal form equation. **For point $q_1 = (3/2, -1, -1/2)$:**

$$v^T q_1 = (1) \left(\frac{3}{2} \right) + (1)(-1) + (-1) \left(-\frac{1}{2} \right) = \frac{3}{2} - 1 + \frac{1}{2} = \frac{4}{2} - 1 = 2 - 1 = 1 \quad (1.44)$$

Since $v^T q_1 = 1 \neq 2$, the point q_1 **does not** lie on the hyperplane H .

For point $q_2 = (5/2, -1, -1/2)$:

$$v^T q_2 = (1) \left(\frac{5}{2} \right) + (1)(-1) + (-1) \left(-\frac{1}{2} \right) = \frac{5}{2} - 1 + \frac{1}{2} = \frac{6}{2} - 1 = 3 - 1 = 2 \quad (1.45)$$

Since $v^T q_2 = 2$, the point q_2 **does** lie on the hyperplane H .

1.3.3 Part c)

Let $K \subset \mathbb{R}^n$ be a non-empty, convex and closed cone, and let $y \notin K$. Since K is closed, we also have $y \notin \overline{K}$. Therefore, we may apply the (strong) separation theorem to the sets $\{y\}$ and K .

By the separation theorem, there exists a vector $v \in \mathbb{R}^n \setminus \{0\}$ such that

$$v^T y < v^T x \quad \forall x \in K. \quad (1.46)$$

1: $v^T y < 0$. Since K is a cone and non-empty, it contains the origin: if $x_0 \in K$, then $0 \cdot x_0 = 0 \in K$. Choosing $x = 0$ in (1.46) gives

$$v^T y < v^T 0 = 0,$$

hence

$$v^T y < 0.$$

2: $v^\top x \geq 0$ for all $x \in K$. Assume, for contradiction, that there exists some $x^* \in K$ with $v^\top x^* < 0$. Because K is a cone, we have $tx^* \in K$ for all $t \geq 0$. Then (1.46) applied to tx^* yields

$$v^\top y < v^\top (tx^*) = t v^\top x^* \quad \forall t \geq 0.$$

Since $v^\top x^* < 0$, the quantity $t v^\top x^*$ tends to $-\infty$ as $t \rightarrow \infty$. Thus we can choose t large enough such that

$$t v^\top x^* < v^\top y,$$

which contradicts the inequality $v^\top y < t v^\top x^*$ above. Therefore, our assumption was false and we must have

$$v^\top x \geq 0 \quad \forall x \in K.$$

Combining Step 1 and Step 2, we have shown that there exists $v \in \mathbb{R}^n \setminus \{0\}$ such that

$$v^\top y < 0 \leq v^\top x \quad \forall x \in K,$$

as required.

1.3.4 Part d)

Statement (c) says that for every point $y \notin K$ there exists a non-zero vector v such that

$$v^\top y < 0 \leq v^\top x \quad \forall x \in K.$$

Geometrically, the equation $v^\top x = 0$ describes a hyperplane through the origin, and the inequalities

$$v^\top x \geq 0, \quad v^\top x < 0$$

describe the two closed and open half-spaces determined by this hyperplane.

Thus, (c) means that for any non-empty, convex and closed cone K and any point $y \notin K$, we can find a hyperplane through the origin such that:

- the entire cone K lies in the closed half-space $\{x \mid v^\top x \geq 0\}$, and
- the point y lies strictly in the opposite open half-space $\{x \mid v^\top x < 0\}$.

In other words, the hyperplane $\{x \mid v^\top x = 0\}$ *strictly separates* the point y from the cone K , and because K is a cone, this separating hyperplane necessarily passes through the origin.

Exercise sheet 8 for Numerical Optimization

Dr. Michael Lehn

Tobias Born

Deadline: December 10, 2025

Exercise 1 (Trust region methods, 8+7+1+1 points)

The last approach for unconstrained optimization we get to know are *trust region methods*. The underlying idea is to approximate the objective function by a quadratic function and then minimize the quadratic function. Suppose the k -th iterate x_k , then Taylor's expansion yields the quadratic approximation

$$f(x_k + p) \approx q_k(p) := f(x_k) + \nabla f(x_k)^T p + \frac{1}{2} p^T H_f(x_k) p . \quad (1)$$

However, this approximation can only be reasonable in a neighbourhood around x_k . Therefore, we define a *trust region*

$$R_k := \{p \in \mathbb{R}^n : \|p\|_2 \leq \Delta_k\}$$

with radius $\Delta_k > 0$ in which we assume the approximation to be good enough. In order to ensure reasonability of the solution, we only minimize (1) inside this trust region. The resulting *trust region subproblem* is

$$\text{find } p_k \in R_k \quad \text{such that} \quad q_k(p_k) = \min_{p \in R_k} q_k(p) . \quad (2)$$

The next iterate is the optimal step inside the trust region,

$$x_{k+1} = x_k + p_k .$$

How to determine the trust region radii Δ_k ? Suppose p_k is the solution of (2). The reduction p_k means for the quadratic approximation is

$$\text{rq}_k = q_k(0) - q_k(p_k) = f(x_k) - q_k(p_k) = - \left(\nabla f(x_k)^T p_k + \frac{1}{2} p_k^T H_f(x_k) p_k \right) ,$$

the reduction p_k means for the function f is

$$\text{rf}_k = f(x_k) - f(x_k + p_k)$$

and the ratio of both will be denoted as

$$\rho_k = \frac{\text{rf}_k}{\text{rq}_k} .$$

If ρ_k is close to zero, the reduction in the quadratic model is much bigger as the reduction in f . From that, we conclude that the quadratic model is a bad approximation of f at $x_k + p_k$ and that the trust region is too large. Therefore, we reject the step and restart at x_k with a trust region with smaller radius. If ρ is significantly bigger than zero, we accept the step. If ρ_k is even close to one and the step reaches the boundary of the trust region, we consider the quadratic approximation to be too good. That means that with a bigger radius we might still have gotten an acceptable approximation but a larger step. So we probably could have been more efficient. We still accept the step, but increase the trust region radius for the next iteration step.

How to solve the trust region subproblems (2)? Of course, one could try to solve the trust region subproblems (2) exactly. However, except for special cases, this is not trivial and would mean a lot of effort. Besides, we do not know methods to solve such kinds of problems yet (it is an optimization with respect to constraints).

Because of that, often not the trust region subproblem itself is solved (an optimization with respect to the whole of R_k). Instead, only a line search along the steepest descent $-\nabla f(x_k)$ is performed. So the actual subproblem becomes

$$\text{find } p_k \in R_k \cap \text{span}\{-\nabla f(x_k)\} \quad \text{such that} \quad q_k(p_k) = \min_{p \in R_k \cap \text{span}\{-\nabla f(x_k)\}} q_k(p) . \quad (3)$$

We hope that this is a good approximation to the original subproblem. The big advantage is that the solution of (3) can be computed explicitly, it is called *Cauchy point*.

a) Prove that

$$p_k = \begin{cases} -\frac{\Delta_k}{\|\nabla f(x_k)\|_2} \nabla f(x_k) & \text{if } \nabla f(x_k)^T H_f(x_k) \nabla f(x_k) \leq 0 \\ \max\left(-\frac{\|\nabla f(x_k)\|_2^2}{\nabla f(x_k)^T H_f(x_k) \nabla f(x_k)}, -\frac{\Delta_k}{\|\nabla f(x_k)\|_2}\right) \nabla f(x_k) & \text{if } \nabla f(x_k)^T H_f(x_k) \nabla f(x_k) > 0 \end{cases}$$

solves (3).

The resulting algorithm is:

Algorithm 1: Trust region method with Cauchy step

```

1 Input:  $f \in C(\mathbb{R}^n, \mathbb{R})^2$ , starting point  $x_0 \in \mathbb{R}^n$ , termination tolerance  $tol > 0$ , initial
   trust region radius  $\Delta_0 > 0$ , parameters  $0 < \eta_1 < \eta_2 < 1$  and  $0 < \nu_1 < 1 < \nu_2$ 
2 Output: approximation  $x_k$  of a stationary point
3  $k \leftarrow 0$ 
4 while  $\|\nabla f(x_k)\|_2 \geq tol$  do
5   // determine step
6   if  $\nabla f(x_k)^T H_f(x_k) \nabla f(x_k) \leq 0$  then
7      $p_k \leftarrow -\frac{\Delta_k}{\|\nabla f(x_k)\|_2} \nabla f(x_k)$ 
8   else
9      $p_k \leftarrow \max\left(-\frac{\|\nabla f(x_k)\|_2^2}{\nabla f(x_k)^T H_f(x_k) \nabla f(x_k)}, -\frac{\Delta_k}{\|\nabla f(x_k)\|_2}\right) \nabla f(x_k)$ 
10  // accept or reject step
11   $\rho_k \leftarrow \frac{f(x_k) - f(x_k + p_k)}{-\nabla f(x_k)^T p_k - \frac{1}{2} p_k^T H_f(x_k) p_k}$ 
12  if  $\rho_k > \eta_1$  then
13     $x_{k+1} \leftarrow x_k + p_k$ 
14  else
15     $x_{k+1} \leftarrow x_k$ 
16  // determine new trust region radius
17  if  $\rho_k \leq \eta_1$  then
18     $\Delta_{k+1} \leftarrow \nu_1 \Delta_k$ 
19  else if  $\rho_k \geq \eta_2 \wedge |\Delta_k - \|p_k\|_2| = 0$  then
20     $\Delta_{k+1} \leftarrow \nu_2 \Delta_k$ 
21  else
22     $\Delta_{k+1} \leftarrow \Delta_k$ 
23   $k \leftarrow k + 1$ 

```

b) Create a MATLAB programme `trust_region_cauchy.m` that implements algorithm 1. You should pass to it

- function handles `f`, `grad_f`, `H_f`,
- a starting point x_0 ,
- a termination tolerance `tol` and a maximum number of iteration steps `max_it`,
- an initial trust region radius Δ_0
- and parameters $\eta_1, \eta_2, \nu_1, \nu_2$.

The iteration is supposed to terminate if the norm of the gradient is smaller than `tol` or if the maximum number of iteration steps is reached. Let the programme return two matrices,

- an $(n \times (\text{number of iteration steps} + 1))$ -dimensional matrix containing all iterates
- and an $(n + 1)$ -dimensional vector containing the trust region radii of all iteration steps.

c) The MATLAB programme `test_trust_region_rosenbrock.m` tests your implementation on Rosenbrock's function. Execute the programme and reflect on the results.

d) Do the same for `test_trust_region_himmelblau.m`.

Exercise 2 (Cones, 2+2+3+4 points)

The next major chapter will be optimization under constraints. To do this, we first need to deal with cones. A set $K \subset \mathbb{R}^n$ is called cone, if for every $x \in K$ and $\alpha \geq 0$ it is $\alpha x \in K$.

- a) First, a simple example to illustrate the concept. Consider

$$K = \left\{ \alpha x \mid \alpha \geq 0, x \in S_1, S_1 = \left\{ \beta \begin{pmatrix} -1 \\ 1 \end{pmatrix} + (1 - \beta) \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \beta \in [0, 1] \right\} \right\} \subset \mathbb{R}^2 .$$

Draw S_1 and K .

- b) Find a set $S_2 \subset \mathbb{R}^2$ with $S_2 \neq \mathbb{R}^2$ such that the cone

$$K = \left\{ \alpha x \mid \alpha \geq 0, x \in S_2 \right\} \subset \mathbb{R}^2$$

equals $\mathbb{R}^2$.

- c) Cones do not need to be convex. Find an example of a non-convex cone.

- d) Prove that $K \subset \mathbb{R}^n$ is a convex cone if and only if it holds that

$$\forall x, y \in K \quad \forall \alpha, \beta \geq 0 \quad \text{it is} \quad \alpha x + \beta y \in K .$$

Exercise 3 (Farkas' Lemma (and hyperplanes and separability), 3+3+7+2+7 points)

Farkas' Lemma is a central tool for deriving optimality conditions in optimization with constraints. We want to get to know it and prove it.

To do this, we will first take a look at hyperplanes. A hyperplane $H \subset \mathbb{R}^n$ is a set

$$H = \left\{ x \in \mathbb{R}^n \mid x = p + \sum_{i=1}^{n-1} s_i u_i, \ s_1, \dots, s_{n-1} \in \mathbb{R} \right\}$$

with $p \in \mathbb{R}^n$ and linearly independent directions $u_1, \dots, u_{n-1} \in \mathbb{R}^n$. The direction vectors form an $(n-1)$ -dimensional affine coordinate system.

a) For $n = 3$, consider the hyperplane

$$H = \left\{ x \in \mathbb{R}^3 \mid x = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} + s_1 \begin{pmatrix} \frac{1}{2} \\ 0 \\ \frac{1}{2} \end{pmatrix} + s_2 \begin{pmatrix} 0 \\ -\frac{1}{2} \\ -\frac{1}{2} \end{pmatrix} \right\} . \quad (4)$$

Visualize the hyperplane drawing it by hand, using MATLAB or any other tool. Check (calculating by hand) if the points

$$q_1 = \begin{pmatrix} \frac{3}{2} \\ -1 \\ -\frac{1}{2} \end{pmatrix} \quad \text{and} \quad q_2 = \begin{pmatrix} \frac{5}{2} \\ -1 \\ -\frac{1}{2} \end{pmatrix}$$

lie on H .

Every hyperplane can be represented in normal form

$$H = \{x \in \mathbb{R}^n \mid v^T x = v^T p\} ,$$

where $v \in \mathbb{R}^n \setminus \{0\}$ is a normal vector of the hyperplane.

b) Express the hyperplane (4) in normal form and add the normal vector v to your visualization from exercise a). Note that the normal form is unique up to the length of the normal vector v .

Check again with the normal form, if q_1 and q_2 lie in H .

A hyperplane divides $\mathbb{R}^n$ into two half-spaces, namely

$$\{x \in \mathbb{R}^n \mid v^T x \leq v^T p\} \quad \text{and} \quad \{x \in \mathbb{R}^n \mid v^T x \geq v^T p\} .$$

The intersection of the interiors of both half-spaces is empty, they only meet on the hyperplane. So, every $x \in \mathbb{R}^n$ with $v^T x < v^T p$ lies on one side of the hyperplane and every $x \in \mathbb{R}^n$ with $v^T x > v^T p$ on the other side of the hyperplane.

The **separation theorem** yields a geometrical statement about convex sets. It can be formulated in many variants, we are interested in the following strong variant:

Consider $K \subset \mathbb{R}^n$ convex with $K \neq \emptyset$. Also consider $y \in \mathbb{R}^n$ with $y \notin \overline{K}$. Then the two sets $\{y\}$ and K are separable, meaning that there exists a $v \in \mathbb{R}^n \setminus \{0\}$ such that $v^T y < v^T x \ \forall x \in K$.

We now ask what happens when K is a non-empty, convex and closed cone.

- c) Let $K \subset \mathbb{R}^n$, $K \neq \emptyset$, be a convex and closed cone. Prove using the separation theorem that for every $y \notin K$ there exists a $v \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ such that for all $x \in K$ it holds that $v^T y < 0 \leq v^T x$.
- d) What does the statement in exercise c) mean with regard to non-empty, convex and closed cones, vectors $y \in \mathbb{R}^n$ and the existence of hyperplanes and the position of y and K in the half-spaces?

Farkas' Lemma states the following: Consider a matrix $B \in \mathbb{R}^{k \times n}$ and a vector $d \in \mathbb{R}^k$. Then exactly one of the two statements is true (order relations applied to vectors are meant elementwise):

- The problem

$$Bx = d, \quad x \geq 0$$

has a solution $x \in \mathbb{R}^n$.

- The problem

$$B^T \lambda \geq 0, \quad \lambda^T d < 0$$

has a solution $\lambda \in \mathbb{R}^k$.

- e) Prove Farkas' Lemma using the statement from exercise c).

Hint: First, show that $K = \{Bx \mid x \geq 0, x \in \mathbb{R}^n\} \subset \mathbb{R}^k$ is a non-empty, convex and closed cone. Second, consider the two cases $d \in K$ and $d \notin K$.

Farkas' Lemma looks quite abstract. But it implies, for example, the so-called KKT conditions. These are necessary first-order optimality conditions for optimization under constraints. In other words, they are the generalization of $\nabla f(x) = \mathbf{0}$ when there are constraints. We will deal with them soon.